Exp No	Experiment Name	Date	Signature
1.	working with Numpy arrays	26/7/26	✓
2.	working of pandas Dataframe	26/7/26	✓
3.	Basic scenario based programs using python for datascience	3/8/26	✓
4.	perform data preprocessing on a given dataset	9/8/26	✓
5.	Dataset splitting and cross-validation using Iris Dataset	17/8/26	✓
6.	Linear Regression using the Iris Dataset	17/8/26	✓
7.	KNN classification using Iris Dataset	7/9/26	✓

Expno:	Experiment name	Date	Signature
8.	Implement a classification model using Support Vector machine (SVM) and compute Evaluation metrics	14/9/26	✓
(9)	Implement k-means clustering using iris dataset	14/9/26	✓
(10)	perform Hierarchical clustering and compare Results using iris dataset	21/9/26	✓
11.	Implement Random Forest and Analyze performance	21/9/26	✓
12.	Implement Boosting Techniques and compare Results	28/9/26	✓
13.	Implement a Basic Reinforcement learning model	28/9/26	✓
14.	Build a Neural network using Tensor Flow / keras.	5/10/26	✓

Ex. NO 1

Working with Numpy arrays

Aim: To write a Numpy arrays program to demonstrate the basic array concepts in Jupyter Notebook.

Algorithm:

Step 1: Start the program
Step 2: Import the Numpy library
Step 3: Define the 1-D array, 2-D array
Step 4: Print the memory address, the shape, the data type and perform some basic operations
Step 5: Then create an array using built-in Numpy functions and perform some mathematical functions
Step 6: Print the output
Step 7: Stop the program

version in your system.

code:

```
import numpy as np
print (np.__version__)
```

output:

2.5.1 ✓ o/p verified.

2) Numpy program to print different types of arrays.

one dimensional array

code:

```
import numpy as np
x = np.arange (2,6).reshape(4)
print (x)
```

output

[2 3 4 5]

two dimensional array

code:

```
import numpy as np
x = np.arange (2,10).reshape (2,4)
print (x)
```

(2 → rows, 4 → column)

output:

```
[[2 3 4 5]
 [6 7 8 9]]
```

code:

```
import numpy as np
x = np.arange (24).reshape (4,3,2)
print (x)
```

(4 → separate matrix, 3 → rows, 2 → columns)

```
[[[ 0  1]
  [ 2  3]
  [ 4  5]]

 [[ 6  7]
  [ 8  9]
  [10 11]]]

[[[12 13]
  [14 15]
  [16 17]]

 [[18 19]
  [20 21]
  [22 23]]]
```

o/p Vandhuchu

③ Numpy program to perform Array slicing

code:

```
impaet numpy as np
arr = np. array ([1, 2, 3, 4])
paint (arr[1:3:2])
paint (arr[:3])
paint (arr[::2])
```

begin stop skip for
[start : stop : step]
0:1 1:2 2:3 3:4
[1:3:2]

beggin
staet

steeping

output:

```
[2]
[1 2 3]
[1 3]
```

o/p Vandhuchu

0 1 2 3
1 2 3 4

(0) (2) index (1) → 1 2 3:2
(1) and & POS skip every 2

4. Numpy program to perform trigonometric functions

code:

```
import numpy as np
arr = np. array ([0, 30, 60, 90
]) paint (np. sin(arr))
```

④ print (np.cos(arr))
print (np.tan(arr))

There is no arccos func see we use 1/np.cos

output:

[0. -0.9 -0.3 0.8 -0.8]
[1. 0.15 -0.9 -0.4 -0.5]
[0. -6.4 0.3 -1.9 1.3]

O/P verified.

⑤ [Numpy program to perform trigonometric functions

code:

import numpy as np
arr = np.array] x

5) Numpy program to convert a list of numeric value into a one dimensional Numpy array

code:

import numpy as np
I = [12.23, 13.32, 100, 36.32]
print ("original List:", I) a =)
np.array (I)
print ("one-dimensional Numpy array:", a)

output:

original List: [12.23, 13.32, 100, 16.32]
one-dimensional Numpy array
[12.23 13.32 100. 36.32] O/P Verified

) Numpy program to create an array with values ranging from 12 to 38

code:

```
import numpy as
npx =
np.arrange (12, 38)
print (x)
```

output

o/p verified

[12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30 31 32 33 34 35 36 37]

⑦ Numpy program to reverse an array (first element becomes last)

code:

```
import numpy as
np import numpy
as np x = np. arrange
(12, 38)
print ("original array:")
print(x)
print ("Reverse array:")
x = x[::-1]
print(x)
```

output

original array:

o/p verified

[12 13 14 15 16 17 18 19 20 21 22 23
24 25 26 27 28 29 30 31 32 33 34
35 36 37] Reverse array: [37 36 35
34 33 32 31 30 29 28 27 26 25 24 23 22 21
20 19 18 17 16 15 14 13 12]

⑧ Numpy program to append values to the end of an array

code:

```
import numpy as
np x = [10, 80, 30]
print ("original array:")
print(x)
x=np.append (x,[[40,50,60],[10,80,90]])
print ("After append values to the end of
the array:")
print(x)
```

output:

(10, 80, 30)

[10, 80, 30 40 50 60 10 80 90]

o/p verified

Numpy program to find common values between two arrays

code:

```
import numpy as np
array 1 = np.array([0,10, 80, 40,60])
print ("Array 1:", array 1)
array 2: [10, 30, 40]
print ("Array 2:", array 2)
print ("common values b/w two arrays:")
print (np.intersect1d (array 1, array 2))
```

output

[10 80 30] [0 10 80 40 60]

o/p verified [10, 20, 30, 40]

[10 20 40]

10. Numpy program to find the union of two arrays.

code:

```
import numpy as np
array 1 = np. array ([0,10,20,40,60,80])
print ("Array 1:", array1)
array 2 = [10, 30, 40, 50, 70]
print ("Array 2:", array 2)
print (" unique sorted array of values that
are in either of the two input arrays:")
print(np. union 1d (array 1, array 2))
```

output

```
[0  10  20  40 60 80] → Arr 1
[10, 30, 40, 50, 70] → Arr 2
[ 0 10 20 30 40 50 60 80]
```

o/p verified

(11) Numpy program to concatenate two 2-dimensional arrays

code:

```
Import numpy as np
a = np. array ([[0, 1, 3], [5, 7, 9]])
b = np. array ([[0, 2, 4], [6, 8, 10]])
c = np. ~~array~~ concatenate ((a, b), 1)
print (c)
```

output

```
[[0 1 3 0 2 4]
 [5 7 9 6 8 10]]
```

o/p verified

(8) Write a Numpy program to convert values of centeigrade degrees into Fahrenheit degrees and vice versa

Fahrenheit degree to centigeade degree

code:

```
import numpy as np
fvalues = [ 0, 12, 45.21, 34, 99.91, 32]
F = np. array (fvalues)
print ("values in fahreenheit degrees:")
print (F)
print ("values in centigeade degrees:")
print (np. round ((5* F/9 - 5*32/9), 2))
```

output

```
values in fahreheit degress
[ 0.  12.  45.21  34.  99.91  32. ]
values in centigrade degree
[-17.78  -11.11  7.34  1.11  37.73
  0. ]
```

o/p verified

code:

```
import numpy as np
cvalues = [-17.78, -11.11, 7.34, 1.11, 37.73]
c = np. adday (cvalues)
print ("values in centigeade degdees:")
print (c)
```

```
print ("Values in Fahrenheit degrees:")
print (np.round (c*9/5 + 32), 2))
```

output

Values in centigrade degrees

[-17.78 -11.11 7.34 1.11 37.73]

Values in Fahrenheit in degrees

[-0. 12. 45.21 34. 99.91]

o/p Verified

3) Numpy program to perform arithmetic operations

code:

```
import numpy as np
a=np.arange (9, dtype = np.float_).reshape (3,3)
print ("First array:")
print (a)
b=np.array ([10,20,30])
print ("Second Array:")
print (b)
print (b)
print ("Addition of Arrays:")
print (np.add (a,b))
print ("subtraction of Arrays:")
print (np.subtract (a,b))
print ("Multiplication of Arrays:")
print (np.multiply (a,b))
print ("Division of Arrays:")
print (np.divide (a,b))
```

Output

first array

[[0. 1. 2.]
[3. 4. 5.]
[6. 7. 8.]]

second array

[10 20 30]

Addition of arrays [[10. 21. 32.]
[13. 24. 35.]
[16. 27. 38.]]

subtraction of arrays [[-10. -19. -28.]
[-7. -16. -25.]
[-4. -13. -22.]]

multiplication of arrays [[0. 20. 60]
[30. 80. 150.]
[60. 140. 240.]]

Division of Arrays:

[[0. 0.05 0.06666667
[0.3 0.2 0.16666667]
[0.6 0.35 0.26666667]]

o/p verified

Result:

thus the Numpy array programs was successfully executed and verified.

19

Exp No: 2 Working of pandas DataFrame

Aim: To write a pandas program using sample data frame to perform operations in its dataframe

Algorithm:

1. open an IDE such as Jupyter
2. Import pandas packages
3. create a Dataframe using pandas and extract details of the dataset such as number of datas in each columns, their data types, memory usage etc.
4. perform operations on rows and columns and renaming them
5. selecting, Adding and deleting data on the dataset
6. Handle the missing datas on the Dataframe

sample Dataframe exam_data = {'name':
['Anastasia', 'pima', 'katherine', 'James',
'Emily', 'michael', 'matthew', 'Laura', 'kevin',
'jonas'],
'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5,
np.nan, 8, 19],
'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],

'qualify': ['yes', 'no', 'yes', 'no', 'no',
'yes', 'yes', 'no', 'no', 'yes']}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

Program:

pandas program to create and display a dataframe from a specified dictionary data which has the index labels.

Code:

import pandas as pd — handles the table data
import numpy as np — handles missing values

exam_data = {'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'michael', 'matthew', 'Laura', 'Kevin', 'Jonas'],

'score': [12, 5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],

'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],

'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no', 'yes']}

labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']

df = pd.DataFrame(exam_data, index = labels)

print(df)

output

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no
c	katherine	16.5	2	yes
d	James	NaN	3	no
e	Emily	9.0	2	no
f	michael	20.0	3	yes
g	matthew	14.5	1	yes
h	laura	NaN	1	no
i	kevin	8.0	2	no
j	jonas	19.0	1	yes

② pandas program to display a summary of the basic information about a specified DataFrame and its data.

code:

```
df = pd.DataFrame(exam_data, index=labels)
print("Summary of the basic information
about this DataFrame and its data:")
print(df.info())
```

output

```
<class 'pandas.core.frame.DataFrame'>
Index: 10 entries, a to j
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   name       10 non-null     str
 1   score      8 non-null      float64
 2   attempts   10 non-null     int64
 3   qualify    10 non-null     str
dtypes: float64(1), int64(1), str(2)
memory usage: 400.0+ bytes
```

Pandas program to get the first rows of a given data frame

Code:

```
df = pd.DataFrame(exam_data, index=labels)
print("First three rows of the data frame:")
print(df.iloc[:3])
```

output

First three rows of DataFrame

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no
c	Katherine	16.5	2	yes

Pandas program to get the select the 'name' and 'score' columns from the following data frame

Code:

```
df = pd.DataFrame(exam_data, index=labels)
print("select specific columns:")
print(df[['name', 'score']])
```

output

	name	score
a	Anastasia	12.5
b	Dima	9.0
c	Katherine	16.5
d	James	NaN
e	Emily	9.0
f	Michael	20.0
g	Matthew	14.5
h	Laura	NaN
i	Kevin	8.0
j	Jonas	19.0

5. pandas program to select the rows where the number of attempts in the examination is greater than 2.

Code:

```
df = pd.DataFrame(exam_data, index=labels)
print("Number of attempts in the examination is greater than 2:")
print(df[df['attempts'] > 2])
```

Output

Number of attempts in the examination is greater than 2:

	name	score	attempts	quality
b	Dima	9.0	3	no
d	James	NaN	3	no
f	michael	20.0	3	yes

6. pandas program to count the number of rows and columns of a DataFrame

code

```
df = pd.DataFrame(exam_data, index=labels)
total_rows = len(df.axes[0])
total_cols = len(df.axes[1])
print("Number of Rows:" +str(total_rows))
print("Number of columns:" +str(total_cols))
```

Output

Number of rows: 10
Number of columns: 4

⑦ pandas program to select the rows where the score is missing i.e. NaN

code:

```
df = pd.DataFrame(exam_data, index=labels)
print("Rows where score is missing:")
print(df[df['score'].isnull()])
```

output

	name	score	attempts	qualify
d	James	NaN	3	no
h	Laura	NaN	1	no

⑧ pandas program to append a new row 'k' to dataframe with given values for each column. Now delete the new row and return the original data frame

```
df = pd.DataFrame(exam_data, index=labels)
print("original rows:")
print(df)
print("\n Append a new row:")
df.loc['k'] = ['Suresh', 15.5, 1, 'yes']
print("print all records after insert a new record:")
print(df)
print("\n Delete the new row and display the original rows:")
df = df.drop('k')
print(df)
```

output

original rows:

	name	score	attempts	quality
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no
c	Katherine	16.5	2	yes
d	James	NaN	3	no
e	Emily	9.0	2	no
f	Micheal	20.0	3	yes
g	Matthew	14.5	1	yes
h	Laura	NaN	1	no
i	Kevin	8.0	2	no
j	Jonas	19.0	1	yes

Append a new row:

print all records after insert a new record:

	name	score	attempts	quality
a	anastasia	12.5	1	yes
b	dima	9.0	3	no
c	katherine	16.5	2	yes
d	james	NaN	3	no
e	emily	9.0	2	no
f	micheal	20.0	3	yes
g	matthew	14.5	1	yes
h	laura	NaN	1	no
i	kevin	8.0	2	no
j	jonas	19.0	1	yes
K	suresh	15.5	1	yes

delete the new row and display the original rows:

a	anatasia	12.5	1	yes
b	dima	9.0	3	no
c	katherina	16.5	2	yes
d	james	NaN	3	no
e	emily	9.0	2	no
f	michael	20.0	3	yes
g	matthew	14.5	1	no
h	laura	NaN	1	no
i	kevin	8.0	2	yes
j	jonas	19.0	1	

9) pandas program to sort ... order first by 'name' in descending order then by 'score' in ascending order

code:

```
df = pd.DataFrame(exam_data, index=labels)
print("Original rows.")
print(df)
df.sort_values(by=['name','score'],
ascending=[False, True])
print("Sort the data frame first by
'name' in descending order, then by
'score' in ascending order:")
print(df)
```

output:

Original rows:

	name	score	attempts	quality
a	anastasia	12.5	1	yes
b	dima	9.0	3	no
c	katherine	16.5	2	yes
d	james	NaN	3	no
e	emily	9.0	2	no
f	michael	20.0	3	yes
g	matthew	14.5	1	yes
h	laura	NaN	1	no
i	kevin	8.0	2	no
j	jonas	19.0	1	yes

sort the data frame first by 'name' in descending order, then by 'score' in ~~des~~ ascending order:

a	anastasia	12.5	1	yes
b	dima	9.0	3	no

c	katherine	16.5	2	yes
d	james	NAN	3	no
e	emily	9.0	2	no
f	michael	20.0	3	yes
g	matthew	14.5	1	yes
h	laura	NAN	1	no
i	kevin	8.0	2	no
j	jonas	19.0	1	yes

pandas program to change the name 'James' to 'Suresh' in name column of the data frame

code:

```
df = pd.DataFrame(exam_data, index=labels)
print("Original rows:")
print(df)
print("\n change the name 'James' to 'shyam':")
df['name'] = df['name'].replace('James', 'shyam')
print(df)
```

output

original rows:

	name	score	attempts	quality
a	anastasia	18.5	1	yes
b	dima	9.0	3	no
c	katherine	16.5	2	yes
d	james	NAN	3	no
e	emily	9.0	2	no
f	michael	20.0	3	yes
g	matthew	14.5	1	yes

h	laura	NaN	1	no
i	kevin	8.0	2	no
j	jonas	19.0	1	yes

change the name 'James to 'shyam':

	name	score	attempts	quality
a	anastasia	12.5	1	Yes
b	dima	9.0	3	no
c	katherine	16.5	2	yes
d	shyam	NaN	3	no
e	emily	9.0	2	no
f	micheal	20.0	3	yes
g	matthew	14.5	1	yes
h	laura	NaN	1	no
i	kevin	8.0	2	no
j	jonas	19.0	1	yes

o/p verified

Result

Thus the working of pandas dataframe was executed and verified successfully

EX NO: 3 Basic scenario based programs using python for datascience

① Student marks Analysis program

```
marks = [78, 85, 92, 67, 88, 90, 76]
print ("Highest Mark:", max(marks))
print ("Lowest Mark:", min(marks))
print ("Average Mark:", sum(marks) / len(marks))
```

output

```
Highest mark: 92
Lowest mark: 67
Average mark: 82.285714 28571429
```

② Tempelature Analysis program

```
tempelatude = [30, 32, 31, 29, 35, 34, 33]
print ("Maximum Tempelature", max(temperature))
print ("Minimum Tempelatude", min (tempelature))
print ("Average Tempelatude", sum (tempelatude)
       /len(tempelatude))
```

output

```
Maximum temperature: 35
minimum temperature 29
Average temperature 32.0
```

③ Weekly sales Analysis program

```
sales = [2500, 3000, 2800, 3500, 4000, 3800,
         4200]
print ("Total Sales:", sum(sales))
print ("Average Sales:", sum(sales) /len(sales))
```

total sales: 23800
average sales: 3400.0

④ Even and odd employee IDs peg

```
ids = [101, 102, 103, 104, 105, 106]
even = []
odd = []
for i in ids:
    if i % 2 == 0:
        even.append(i)
    else:
        odd.append(i)
print("Even IDs:", even)
print("odd IDs:", odd)
```

output

```
odd IDs: [101, 103, 105]
Even IDs: [102, 104, 106]
```

output

```
Even IDs: []
Odd IDs: [101]
Even IDs: [102]
```

⑤ count positive and negative values peg

```
values = [20, -5, 15, -10, 30, -2]
positive = 0
negative = 0
for i in values:
    if i > 0:
        positive += 1
    elif i < 0:
        negative += 1
print("positive values:", positive)
print("Negative values:", negative)
```

output

positive values 1
Negative values 1

positive values 2
Negative values 2

positive values 3
negative values 3

⑥ Find duplicate customer IDs prg.

```
customer_ids = [101, 102, 103, 104, 102, 105, 101]
duplicates = []
for i in customer_ids:
    if customer_ids.count(i) > 1 and i not in duplicates:
        duplicates.append(i)
print("Duplicate IDs:", duplicates)
```

output

Duplicate IDs: [102, 101]

⑦ Product price Analysis problem

```
prices = [250, 450, 120, 600, 350]
print("Highest price:", max(prices))
print("Lowest price:", min(prices))
```

output

Highest price: 600
lowest price: 120

⑧ Empolyee salary increament prg

```
salary = [25000, 30000, 40000, 35000]
new_salary = []
for s in salary:
    new_salary.append(s * 1.10)
print("updated salaries:", new_salary)
```

output

updated salaries: [27500.0004, 33000.6, 44000.6, 38500.0]

⑨ Attendance percentage program

```
total_classes = 90
attended = 81
percentage = (attended / total_classes) * 100
print ("Attendance percentage:", percentage
```

output

attendance percentage: 90.0

5) Students who passed program

```
marks = [45, 67, 89, 32, 78, 55, 49]
passed = []
for m in marks:
    if m >= 50:
        passed.append(m)
print ("passed student marks:", passed)
```

output

Passed student marks: [67, 89, 78, 55]

) shopping Bill calculation prog

```
prices = [120, 80, 150, 200]
total = sum(prices)
print ("Total Bill:", total)
```

Output

Total Bill = 550

), word Frequency Analysis prog

```
feedback = "good service good quality
excellent service"
words = feedback.split()
for word in set(words):
print (word, ":", words.count(word))
```

output

excellent : 1
good : 2
service : 2
quality : 1

NOTE: the order of words may vary because set() does not preserve order.

(13) Age group classification peg

```
ages = [18, 25, 45, 67, 15]
for age in ages:
    if age < 18:
        print(age, "child")
    elif age < 60:
        print(age, "Adult")
    else:
        print(age, "Senior citizen")
```

output

```
12 child        67 senior citizen
25 adult        15 child
45 adult
```

(14) Electricity Bill calculation peg

```
units = 850
if units <= 100
    bill = units * 2
elif units <= 200
    bill = 100 * 2 + (unit - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
print("Electricity Bill:", bill)
```

output

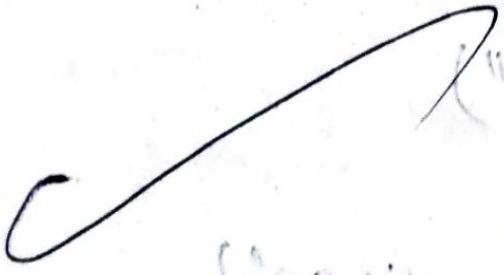
Electricity Bill : 750

(15). monthly Expense Analysis peg

```
expenses = [12000, 15000, 18000, 11000, 17000]
highest = max(expenses)
month = expenses.index(highest) + 1
print("Highest Expense:", highest)
print("Month:", month)
```

output

Highest expenses : 11000
month : 4



o/p verified

Exp No: 4 Perform data preprocessing on given dataset

Aim:
To perform data preprocessing on a given IRIS dataset.

Steps:
1. Loaded the Iris dataset
2. Displayed the first five records
3. checked the dataset structure using info()
4. checked for missing values (none where found in this dataset)
5. Removed the ID column.
6. converted the categorical species column into numerical values using LabelEncoder.
7. standardized the numerical features using standard scaler

Program

```
# import libraries
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Load dataset
df = pd.read_csv("Iris.csv")
# Display first 5 records
print("First 5 Records:")
print(df.head())
# check for missing values
# Display dataset information
```

```
print("\n Dataset Information:")
print(df.info())
# check for missing values
print("\n missing values:")
print(df.isnull().sum())
# Remove the Id column (not useful for
  analysis)
df = df.drop("Id", axis=1)
# Encode the target column (species)
encoder = LabelEncoder()
df["species"] = encoder.fit_transform
                (df["species"])
print("\n Encoded species:")
print(df.head())
# feature scaling
scaled = standard scaler()
features = ["sepal Length cm", "sepal width
  cm", "petal length cm", "petal width cm"]
df[features] = scaler.fit_transform
               (df[features])
print("\n preprocessed dataset:")
print(df.head())
```

output

First 5 record:

	Id	SepalLengthcm	Sepalwidthcm	Petal lengthcm	Petal width cm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Dataset information

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Id              150 non-null    int64
 1   SepalLengthcm   150 non-null    float64
 2   SepalWidthcm    150 non-null    float64
 3   petalLengthcm   150 non-null    float64
 4   petalWidthcm    150 non-null    float64
 5   Species         150 non-null    str
dtypes: float64(4) int64(1), str(1)
memory usage: 7.2+KB
None
```

O/P verified

missing value

```
Id:              0
SepalLengthcm :  0
SepalWidthcm :   0
petalLengthcm    0
petalWidthcm:    0
```

dtype : int64

Encoded species:

	Sepal length cm	Sepal width cm	petal length cm	petal width cm	Species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Prepreocessed Dataset

	Sepal length cm	Sepal width cm	Petal length cm	petal width cm	Spe
0	-0.9	1.03	-1.34	-1.31	0
1	-1.14	-0.12	-1.34	-1.31	0
2	-1.38	0.33	-1.39	-1.31	0
3	-1.50	0.10	-1.28	-1.31	0
4	-1.02	1.26	-1.34	-1.31	0

5. Dataset splitting and cross - Validation using Iris Dataset

this program demonstrates two important techniques in machine learning

① Dataset splitting: dividing the dataset into training and testing sets

② k-Fold cross Validation: Evaluating the model on multiple train-validation splits.

objective

To implement dataset splitting and k-fold cross validation using the Iris dataset and evaluate the performance of a machine learning model

① complete python program

```
# Dataset splitting and cross-validation
# using Iris Dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, kFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

#
# Step 1 Load the Iris dataset
#
iris = load_iris()
x = iris.data
y = iris.target
print("Dataset shape:", x.shape)
print("Number of samples:", len(x))
print("Number of Features:", x.shape[1])
print("Target classes:", iris.target_names)
```

```
# Step 2: split Dataset
Testing
#
X_train, X_test, y_train, y_test = train_test
split ( x, y, test_size = 0.20, random_state
stratify = y)
print ("\n Dataset splitting")
print ("----------")
print (" Training samples:", len(X_train))
print (" Testing samples:", len(X_test))
#
# Step3: create Logistic Regression Model
#
model = LogisticRegression (max_iter = 200)
# Train the model
model.fit (X_train, y_train)

#
# Step4. predict Test Data
#
y_pred = model. predict (X_test)

# calculate test accuracy
accuracy = accuracy_score (y_test, y_pred)
print ("\n Test Set performance")
print ("----------")
print (" Test Accuracy:", round (accuracy, 4
```

```
#
step5. Implement 5-Fold cross validation
# ---------- ---

kf = KFold(
    n_splits = 5
    shuffle = True,
    random_state = 42
)
cv_scores = cross_val_score(model, x, y,
cv=kf, scoring = 'accuracy')
print("\n 5-fold cross validation")
print("----------")
for i, score in enumerate(cv_scores, start=1):
    print("fold", i, "Accuracy:", round(score,4))

#
step6. Calculate Average cross-validation score
#
    print("\n cross-validation Results")
    print("----------")
    print("Average Accuracy:", round(cv_scores.
mean(), 4))
    print("standard deviation:", round(
    cv_scores.std(), 4))
```

output

```
Data shape: (150, 4)
Number of samples: 150
Number of features: 4
Target classes ['setosa' 'versicolor' 'virginca']

Dataset splitting
- - - - - -
Training samples: 120
Testing samples : 120

test set performance
- - - - - -
Total accuracy: 0.9667

5-Fold cross-validation
- - - - - -
Fold 1 Accuracy : 1.0
Fold 2 Accuracy: 1.0
Fold 3 Accuracy: 0.9333
Fold 4 Accuracy: 0.9667
Fold 5 Accuracy: 0.9667

cross-validation
- - - - - -
Average Accuracy: 0.9733
Standard deviation: 0.9733
```

O/P verified

EXNO: 6. Linear Regression using the Iris Dataset

① Python program

```
# Linear Regression using Iris Dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error,
    mean_absolute_error, r2_score
import numpy as np

#
# Step 1: Load Iris Dataset
#
iris = load_iris()
x = iris.data
feature_names = iris.feature_names
# predict petal length
# column 2 = petal Length
Y = iris.data[:,2]
# use sepal length, sepal width and petal width
# as input features
X = iris.data[:,[0,1,3]]
print("features:")
print(feature_names)
print("\n Dataset shape:", X.shape)
print("Target Shape:", y.shape)
```

```
#
# Step 2. Split Dataset
#
X_train, X_test, y_train, y_test = train_test_
    X, y, test_size = 0.20, random_state = 42)
print("In Training samples:", len(X_train))
print("Testing samples:", len(X_test))
#
Step 3. Create linear Regression model
#
model = LinearRegression()
#
#Step 4 Train the model
#
model.fit(X_train, y_train)
#
# Step 5 - make predictions
#
y_pred = model.predict(X_test)
#
#Step 6 - Display predictions
#
print("In Actual vs predicted values")
print("-----")
for actual, predicted in zip(y_test[:10],
                             y_pred[:10]):
    print("Actual:", round(actual, 2),
          "predicted:", round(predicted, 2))
#
Step 7: Evaluate the model
```

```
mse = mean_squared_error (y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error (y_test, y_pred)
r2 = r2_score (y_test, y_pred)
print("\n Model Evaluation")
print("-----------")
print("Mean squared Error (MSE):", round(mse,4))
print("Root mean Squared Error (RMSE):",
                    round (rmse, 4))
print("Mean Absolute Error (MAE):", round(
                    mae,4))
print("R2 score:", round (r2,4))

# Step 8: Display model coefficients
#
print ("\n Model coefficients")
print("-----")
for name, coefficient in zip( ["Sepal Length", "Sepal width", "Petal
                    width"],
    model.coef_):
    print(name, ":", round (coefficient, 4)
print ("Intercept:", round (model.intercept_,4))
```

Expected output

the exact numerical values can vary slightly depending on the train-test split, but the output will be similar to:

Features
['sepal length (cm)' 'sepal width (cm)'
'petal length (cm)' 'petal width (cm)']

Dataset shape : (150, 3)
Target shape : (150,)

Training samples : 120
Testing Samples : 30

Actual vs predicted values

Actual 5.1 Predicted : 5.1
Actual 4.0 Predicted : 4.2
Actual 1.3 Predicted : 1.5
Actual 1.5 Predicted : 1.4
Actual 4.7 Predicted : 4.6

Model Evaluation

Mean squared Error (MSE) : 0.193
Root Mean Squared Error (RMSE) : 0.439606
Mean Absolute Error (MAE) : 0.36
R2 score : 0.9603

Model coefficients

---

sepal length : 0.7228
sepal width : -0.6358
petal width : 1.4675
Intercept : -0.2622

O/P Verified

KNN classification using Iris Dataset experiment no:7

Aim: Implement a classification model using KNN Python program

```
# KNN classification using Iris dataset

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbours import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score )

# ----------
# Step 1: Load Iris Dataset
# ----------

iris = load_iris()
X = iris.data
y = iris.target
print("Dataset shape:", X.shape)
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 2: split Dataset

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
```

```
print("\n dataset splitting")
print("-------")
print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))

#
# Step 3: Feature scaling
# ----------
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
x_test = scaler.transform(x_test)
# Step 4: Create KNN model
# ----------
knn = KNeighborsClassifier(n_neighbors=5)
# ----------
# Step 5: Train the model
# ----------
knn.fit(X_train, y_train)
# Step 6: Make predictions
y_pred = knn.predict(x_test)
# ----------
# Step 7: Calculate Evaluation metrics
# ----------
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
precision = precision_score(
    y_test
    y_pred,
    average = 'weighted'
)
recall = recall_score(
    y_test,
    y_pred,
    average = 'weighted'
)
```

```
    recall = recall_score(
    y_test
    y_pred
    average = 'weighted'
)
    f1 = f1_score(
    y_test,
    y_pred,
    average = 'weighted'
)
# step 8: Display Results
#
print("\nKNN Model Evaluation")
print("-----------")
print("Accuracy:", round(accuracy, 4))
print("precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))
print("\n confusion matrix")
print("-----")
print(cm)
```

Expected output

The exact values may vary slightly depending on the software environment output will be similar to:

Dataset shape: (150, 4)

Number of samples: 150

Number of features: 4

Target classes: ['setosa' 'versicolor' 'virginica']

Dataset splitting

Training samples: 120

Testing samples: 30

KNN model Evaluation

Accuracy: 0.9333

Precision: 0.9444

Recall: 0.9333

F1-score: 0.9327

confusion matrix

[[10 0 0

[0 10 0]

[0 2 8]]

O/P verified

The attached laboratory document also uses an 80% training and 20% testing split. giving 120 training samples and 30 testing samples.

Result: Thus, the K-Nearest Neighbors (KNN) classification model was successfully implemented using the iris dataset and its performance was evaluated using accuracy, precision, F1-score

SVM classification using Iris Dataset

Experiment No: 8

Title

Implement a classification model using support vector machine (SVM) and compute evaluation metrics

Python program

```
# SVM classification using Iris dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)
#
Step 1: Load Iris dataset
#
iris = load_iris()
X = iris.data
y = iris.target
print("Dataset shape:", X.shape)
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)
#
# Step 2: Split dataset
#
```

```
X_train, X_test, y_train, y_test =
train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=48,
    stratify=y
)
print("\n Dataset splitting")
print("-----")
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step3: Feature Scaling

scaler = StandardScaler().
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 4: Create SVM model

model = SVC(
    kernel='linear',
    C=1.0
)
# Step5: Train the model

model.fit(X_train, y_train)

# Step 6: Make predictions

y_pred = model.predict(X_test)

# Step 7: Calculate Evaluation metrics
```

```
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
precision = precision_score(
    y_test,
    y_pred,
    average = 'weighted'
)
recall = recall_score(
    y_test,
    y_pred,
    average = 'weighted'
)
f1 = f1_score(
    y_test,
    y_pred
    average = 'weighted'
)
```

step8 : Display Results

```
print("\nSVM Model Evaluation")
print("-----")
print("Accuracy:", round(accuracy,4))
print("precision:", round(precision,4))
print("Recall:", round(recall, 4))
print("F1-score:", round(f1, 4))
print("\nconfusion matrix")
print("-----")
print(cm)
```

Expected output

The exact values may vary slightly depending on the execution environment

Dataset shape (150, 4)
Number of samples: 150
Number of features: 4
Target classes: ['setosa' 'versicolor' 'virginica']

Dataset splitting

Training samples: 120
Testing samples: 30

SVM Model Evaluation

Accuracy: 1.0
precision: 1.0
Recall: 1.0
F1-score: 1.0
confusion matrix

[[10 0 0]
[0 10 0]
[0 0 10]]

op verified

Result

Thus, the SVM classification model was successfully implemented using the Iris dataset, and its performance was evaluated using Accuracy, confusion Matrix, precision, Recall and F1-score

Experiment No: 9

Implementation of K-mean clustering using Iris datasets.

Title

Implement k-means clustering using Iris Dataset

Python program

```
# K-Means clustering using Iris dataset
from sklearn.datasets import load_iris
from sklearn.cluster import kmeans
from sklearn.preprocessing import standardscaler
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# ---------
# Step 1: Load Iris dataset
# ---------
iris = load_iris()
x = iris.data
y = iris.target
print("Dataset shape:", x.shape)
print("Number of samples:", x.shape[0])
print("Number of features:", x.shape[1])
print("\n features:")
print(iris.feature_names)

# ---------
# Step 2. Feature scaling
# ---------
scaler = standardscaler()
x_scaled = scaler.fit_transform(x)

# ---------
# Step 3: Create K-means model
# ---------
kmeans = kMeans(
    n_clusters = 3
    random_state = 42,
    n_init = 10
```

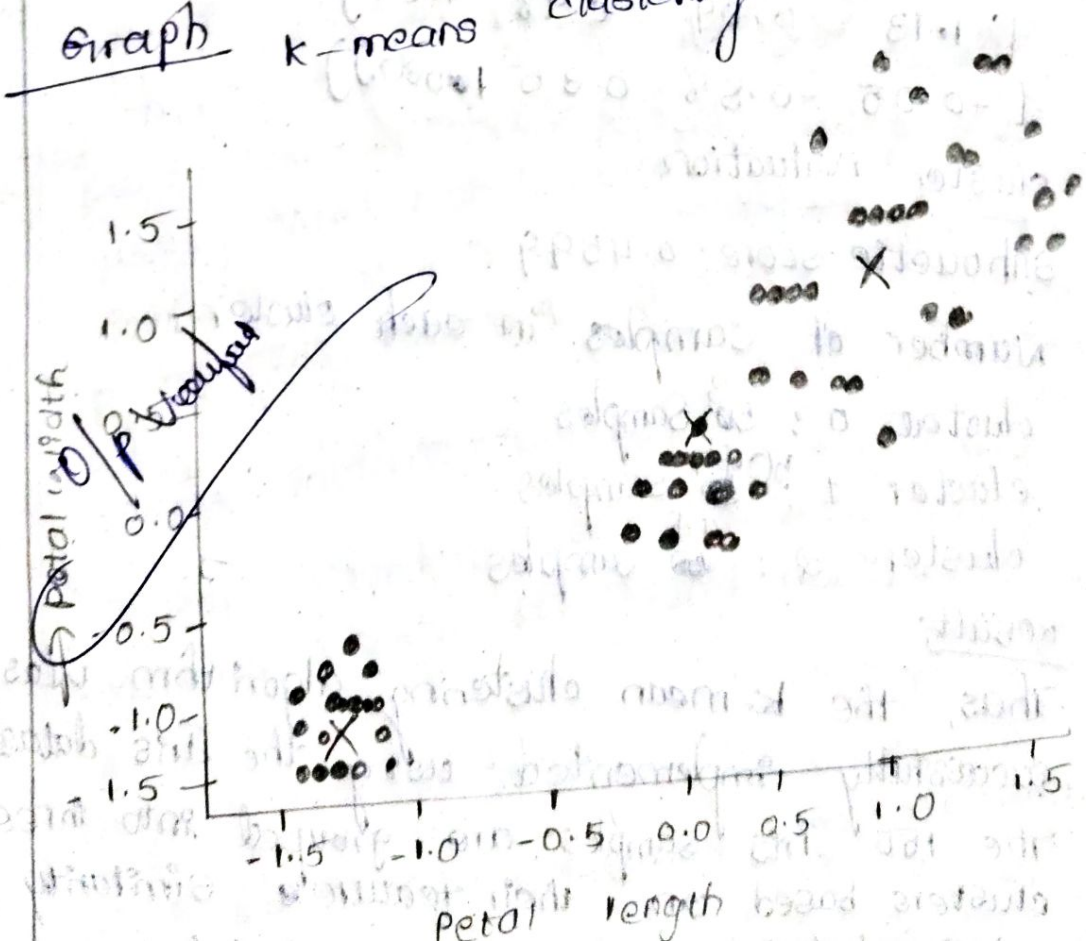
```
# Step 4: Fit k-means model
kmeans.fit(x_scaled)
# Step5 Get cluster labels
labels = kmeans.labels_
# Step6: Get cluster centroids
centroids = kmeans.cluster_centers_
# Step7 : Display Results
print("\n cluster Labels:")
print(labels)
print("\n In cluster centroids:")
print(centroids)
# Step8 : Calculate silhouette score
silhouette = silhouette_score(x_scaled, labels)
print("\n cluster Evaluation")
print("-----")
print("silhouette score:",
      round(silhouette,4))
# Step 9: Display cluster count
print("\n Number of samples in Each cluster")
for i in range(3)
    print(
        "cluster", i,
        ":",
        sum(labels==i),
        "samples"
```

)

#Step 10: visualize clusters

```
plt.figure(figsize=(8,6))
plt.scatter(
    X_scaled[:,0],
    X_scaled[:,3],
    c=labels
)
plt.scatter(
    centroids[:,0]
    centroids[:,3]
    marker='X',
    s=200
)
plt.xlabel("petal length")
plt.ylabel("petal width")
plt.title("K-means clustering on Iris dataset")
plt.show()
```

Graph K-means clustering on Iris datasets



Expected output

The exact cluster numbers may vary because k-means assigns cluster labels automatically.

A typical output is:

```
Dataset shape: (150,4)
Number of samples: 150
Number of features: 4
Features:
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
Cluster labels:
[1 1 1 1 1 1 1 1 1 1 ...]
Cluster centroids:
[[-0.99  0.87 -1.30 -1.25]
 [ 1.13  0.09  0.98  1.01]
 [-0.05 -0.88  0.35  1.0036]]
```

Cluster Evaluation

Silhouette score: 0.4599

Number of samples in each cluster:

cluster 0: 53 samples
cluster 1: 50 samples
cluster 2: 47 samples

Result:

Thus, the k-mean clustering algorithm was successfully implemented using the Iris dataset. The 150 Iris samples are grouped into clusters based

Hierarchical clustering using Iris Dataset

Experiment NO: 10

Title

perform Hierarchical clustering and compare Results using Iris Dataset

Python program

```
# Hierarchical clustering using Iris dataset
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import standardscaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
# from scipy.cluster.hierarchy import dendrogram, linkage
# step 1: Load Iris data
#
iris = load_iris()
x = iris.data
y = iris.target
print("Dataset shape:", x.shape)
print("Number of samples:", x.shape[0])
print("Number of Features:", x.shape[1])
print("\n features")
print(iris.feature_names)
# step 2: feature scaling
scaler = StandardScaler()
x_scaler = scaler.fit_transform(x)
#
step 3: create program
```

```
plt.figure(figsize=(10,6))
dendrogram(
    linked,
    truncate_mode='lastp',
    p=20
)
plt.title("Hierarchial clustering Dendrogram")
plt.xlabel("Data points) clusters")
plt.ylabel("Distance")
plt.show()
# Step4: Apply Agglomerative clustering
model = AgglomerativeClustering(
    n_clusters=3
    linkage='ward'
)
labels = model.fit_predict(x_scaled)
# step5: Display cluster labels
print("In cluster Labels:")
print(labels)
# step6: calculate silhouette score
silhouette = silhouette_score(
    x_scaled,
    labels
)
print("In silhouette Score:",
    round(silhouette,4))
# Step7: Count Samples in Each cluster:")
```

```
print("\n Number of samples in Each
 cluster: ")
for i in range(3)
    print(
        "cluster", i,
        ":",
        sum(labels == i),
        "samples"
    )
# Step8: visualize clusters
plt.figure(figsize=(8, 6))
plt.scatter(
    X_scaled[:, 2],
    X_scaled[:, 3],
    c = labels
)
plt.xlabel("petal Length")
plt.ylabel("petal width")
plt.title("Hierarchical clustering on Iris Dataset")
plt.show()
```

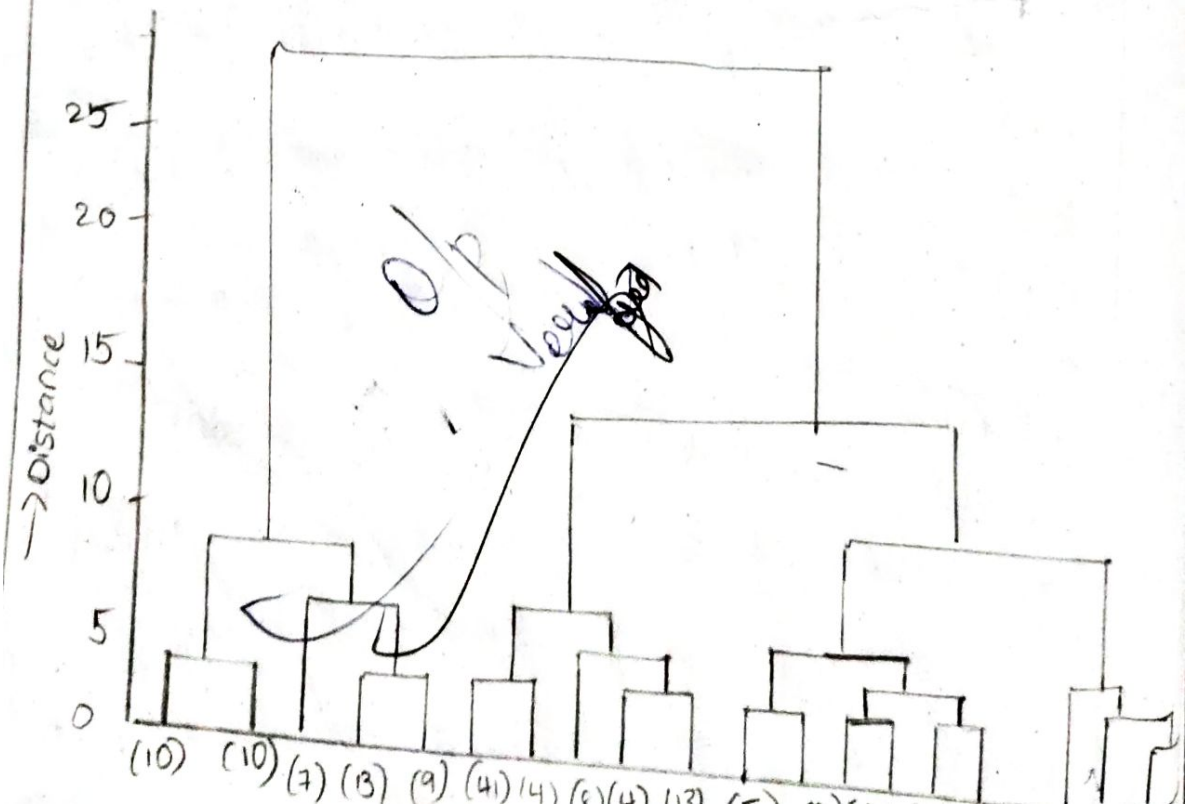
Expected output

A typical output is:

```
Dataset shape: (150, 4)
Number of samples: 150
Number of features: 4

Features:
['sepal length (cm)',
 'sepal width(cm)',
 'petal length(cm)',
 'petal width(cm)']
cluster labels:
  [1, 1 1 1 1 1 1 1 1 ...]
Silhouette score: 0.4467
Number of samples in Each cluster:
cluster 0: 71 samples
cluster 1: 49 samples
cluster 2: 30 samples
```

Hierarchial clustering Dendrogram



Implement Random Forest and Analyze performance

Experiment - 11

python program

```
# Random forest classification using Iris Dataset
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
import matplotlib.pyplot as plt
import seaborn as sns
# 1. Load iris dataset
iris = load_iris()
X = iris.data
y = iris.target
# 2. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
# 3. Create Random Forest model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
```

```
# 4. Train the model
model.fit(X_train, y_train)
# 5. Make predictions
y_pred = model.predict(X_test)
# 6. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Random Forest Classification")
print("-------------------")
print("Number of trees:", 100)
print("Accuracy:", accuracy)
# 7. Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("\n Confusion matrix:")
print(cm)
# 8. Classification Report
print("\n Classification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names = iris.target_names
))
# 9. Display Confusion matrix
plt.figure(figsize=(6,4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels = iris.target_names,
    yticklabels = iris.target_names
)
```

```
plt.xlabel("predicted class")
plt.ylabel("Actual class")
plt.title("Random forest - confusion Matrix")
plt.show()
```

output

the exact values can vary slightly depending on the train-test split and parameters

Random forest classification

Number of trees: 100

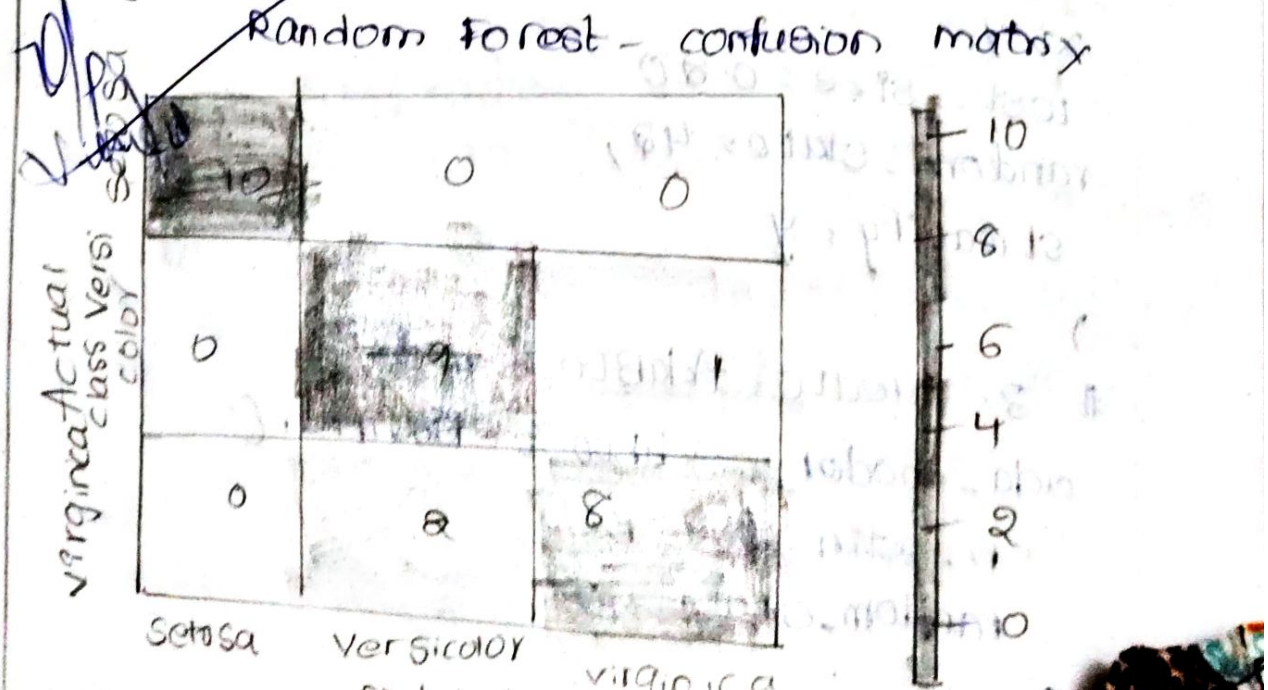
Accuracy: 0.9666666666667

confusion matrix:

```
[[10  0  0]
 [ 0 29  1]
 [ 0  2  8]]
```

classification Report:

	precision	recall	f1-score	support
Setosa	1.00	1.00	1.00	10
Versicolor	0.89	0.90	0.86	10
virginica	0.89	0.80	0.84	10



Implement Boosting Techniques and compare Results Experience-12

Python program

```
# Implement Boosting Technique and
compare Results
# Dataset : Iris
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_
from sklearn.ensemble import AdaBoostClassifi
GradientBoostingClassifier
from sklearn.metrics import accuracy_score
precision_score, recall_score, f1_score
import matplotlib.pyplot as plt
# 1. Load dataset
iris = load_iris()
X = iris.data
y = iris.target
# 2. Split dataset
X_train, X_test, y_train, y_test =
train_test_split(
X, y
test_size = 0.20
random_state = 48,
stratify = y
)
# 3. Create AdaBoost model
ada_model = AdaBoostClassifier(
n_estimators = 100,
random_state = 48
)
```

```
# 4. create Gradient Boosting model
gb_model = GradientBoostingClassifier(
    n_estimators = 100,
    random_state= 42
)
# 5 Train models
ada_model.fit(x_train, y_train)
gb_model.fit(x_train, y_train)
# 6. predictions
ada_pred = ada_model.predict(x_test)
gb_pred = gb_model.predict(x_test)
# 7. function to calculate performance
def evaluate_model(name, y_test, y_pred):
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(
        y_test, y_pred, average = "weighted"
    )
    recall = recall_score(
        y_test, y_pred, average = "weighted"
    )
    f1 = f1_score
        y_test, y_pred, average = "weighted"
    )
    print("\n", name)
    print("-" * 30)
    print("Accuracy:", round(accuracy, 4))
    print("precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("f1_score:", round(1, 4))
    return accuracy, precision, recall, f1
```

```
# 8. Evaluate AdaBoost
ada_results = evaluate_model(
    "AdaBoost",
    y_test
    ada_pred
)
# 9. Evaluate Gradient Boosting
gb_results = evaluate_model(
    "Gradient Boosting",
    y_test
    gb_pred
)
# 10. compare accuracy
models = ["AdaBoost", "Gradient Boosting"]
accuracies = [
    ada_results[0],
    gb_results[0]
]
plt.figure(figsize=(7,5))
plt.bar(models, accuracies)
plt.xlabel("Boosting Techniques")
plt.ylabel("Accuracy")
plt.title("Comparison of Boosting Techniques")
plt.ylim(0,1.1)
plt.show()
```

Sample output

The exact values may vary slightly depending on the train_test split and scikit-learn version.

AdaBoost

Accuracy: 0.9667

precision: 0.9697

Recall: 0.9667

F1_score: 0.9666

Gradient Boosting

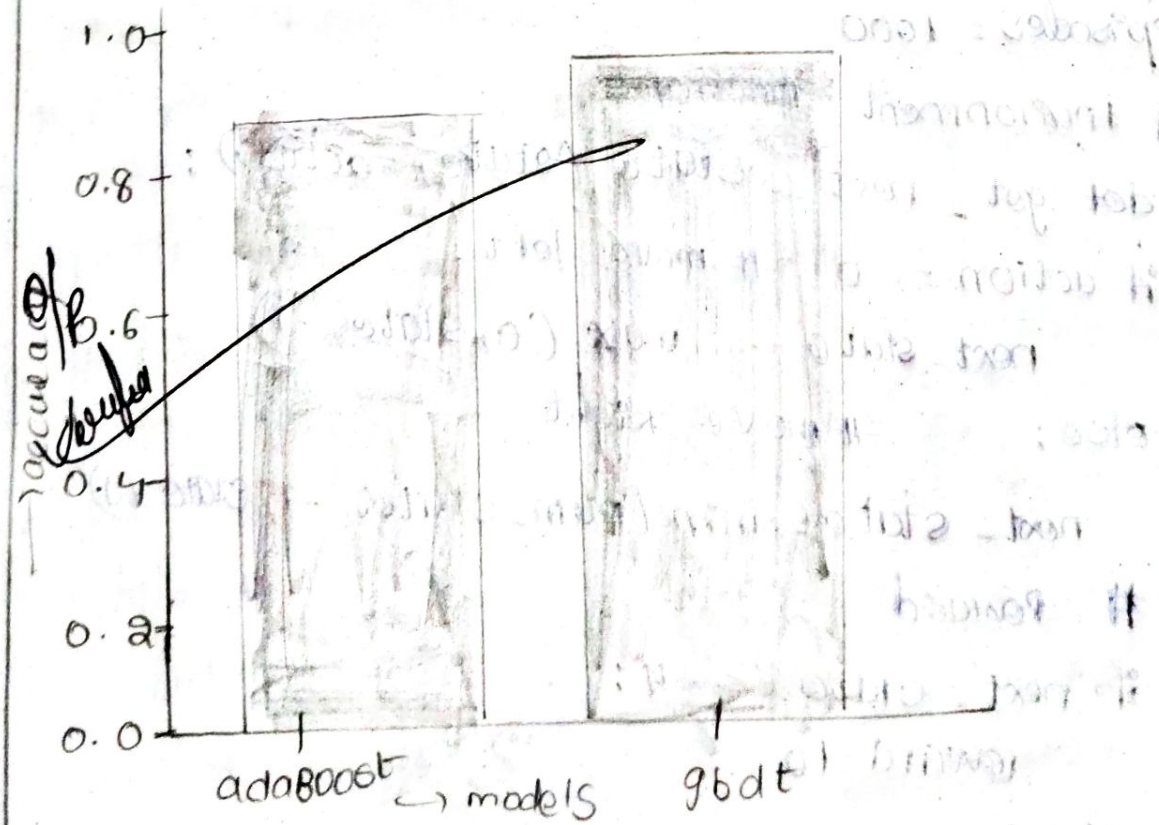
Accuracy: 0.9667

precision: 0.9697

Recall: 0.9667

F1_score: 0.9666

AdaBoost vs Gradient Boosting models



Implement a Basic Reinforcement learning model Experience - 13

Python program

```
import numpy as np
import random
# Number of states and actions
num_states = 5
num_actions = 2
# Actions
# 0 = left
# 1 = Right
# Initialize Q-table
Q = np.zeros((num_states, num_actions))
# parameters
alpha = 0.1 # learning rate
gamma = 0.9 # discount factor
epsilon = 0.8 # Exploration probability
episodes = 1000
# Environment function
def get_next_state(state, action):
if action == 0: # move left
    next_state = max(0, state - 1)
else:       # move Right
    next_state = min(num_states - 1, state + 1)
# Reward
if next_state == 4:
    reward = 10
else:
    reward = -1
return next_state, reward
```

```
# Q-learning
for episode in range(episodes):
    state = 0
    while state != 4:
        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0,1)
        else:
            action = np.argmax(Q[state])
        # perform action
        next_state, reward = get_next_state(state, action)
        # Q-learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward +
            gamma * np.max(Q[next_state]) -
            Q[state, action]
        )
        state = next_state
# Display Q-table
print("Learned Q-table:")
print(Q)
# Test the learned policy
state = 0
path = [state]
while state != 4:
    action = np.argmax(Q[state])
    next_state, reward = get_next_state
                         (state, action)

    state = next_state
    path.append(state)
```

```
print ("\n learned path:")
print (path)
print("\n optimal policy:")
for state in range (num_states-1):
    action = np.argmax (Q[state])
if action == 0:
    print ("state", state, "→ left")
else:
    print ("state", state, "→ right")
```

Sample output

```
learned Q-table:
[[5.561  7.890]
 [5.561  8.100]
 [7.890  9.000]
 [8.100  10.000]
 [0.000  0.000]]
learned path:
 [0, 1, 2, 3, 4]
optimal policy:
state 0 → Right
state 1 → Right
state 2 → Right
state 3 → Right
```

o/p snapshot

Build a Neural Network using Tensorflow / keras

Python program

```
# Build a Neural Network using Tensorflow / keras
# dataset.Iris
import numpy as np
import tensorflow as tf
from tensorflow.keras models import Sequential
from tensorflow.keras layers import Dense
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score,
    classification_report
# 1. Load iris dataset
iris = load_iris()
x = iris.data
y = iris.target
# 2. split dataset
x_train, x_test, y_train, y_test = train_test
                                    - split(
    x, y,
    test_size = 0.80,
    random_state = 42,
    stratify = y
)
# 3. standardize features
scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

```
# 4. Build neural network
model = Sequential([
    Dense(16, activation='relu', input_
shape=(4,)),
    Dense(8, activation='relu'),
    Dense(3, activation='softmax')
])
# 5. compile model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# 6. Display model architecture
model.summary()
# 7. Train the model
history = model.fit(
    x_train,
    y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.2,
    verbose=1
)
# 8. Evaluate model
loss, accuracy = model.evaluate
    (x_test, y_test, verbose=0)
print("\nTest Loss:", round(loss,4))
```

```
print("Test Accuracy:", round(accuracy, 4))
# 9. Make predictions
y_probability = model.predict(X_test, verbose=0)
y_pred = np.argmax(y_probability, axis=1)
# 10. classification report
print("\n Classification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names = iris.target_names ))
```

Sample output

```
Model: "sequential"
Layer (type)          Output Shape          Param #
dense (Dense)         (None, 16)            80
dense_1 (Dense)       (None, 8)             136
dense_2 (Dense)       (None, 3)             27
Total params: 243
Example:
Test loss: 0.0865
Test Accuracy: 0.9667
Example classification result
              precision    recall  f1-score
setosa            1.00      1.00      1.00
versicolor        0.91      1.00      0.95
virginica         1.00      0.90      0.95
Epoch 1/100
3/3 ——— 1s 83ms/step - accuracy:
0.4871 - loss: 1.0840 - val_accuracy:
0.4167 - val_loss: 1.0471
```

Epoch 2/100

3/3 ——— 0s 25ms/step - accuracy: 0.4688 - loss: 1.0562 - val_accuracy: 0.4583 - val_loss: 1.0241

Epoch 3/100

3/3 ——— 0s 23ms/step - accuracy: 0.3333 - loss: 1.0300 - val_accuracy: 0.4583 - val_loss: 1.0024

Epoch 4/100

3/3 ——— 0s 23ms/step - accuracy: 0.5625 - loss: 1.0046 - val_accuracy: 0.4583 - val_loss: 0.9818

Test loss: 0.3336

Test accuracy: 0.8

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.42	0.57	10
2	0.62	1.00	0.72	10
accuracy			0.80	30
macro avg	0.88	0.86	0.78	30
weighted avg	0.88	0.80	0.78	30

Result